

Lecture 5: Convolutional Neural Networks

Today:

00:00) Expectations of visual recognition systems

00:15) A Conv layer, stride, padding etc.

01:35) Conv2D

00:45) ---- Break ----

01:00) Receptive field and intuition

01:15) Transposed conv

01:20) Implementation and backprop

01:45) ---- Break ----

02:00) Max-pooling, convnet and conv variants

02:20) Visualizing and understanding

02:45) ---- Break ----

03:00) Tutorial: Adversarial examples



Monitor

Desk

Mouse

vertical
line

keyboard

key key key

key key key

key key

key key

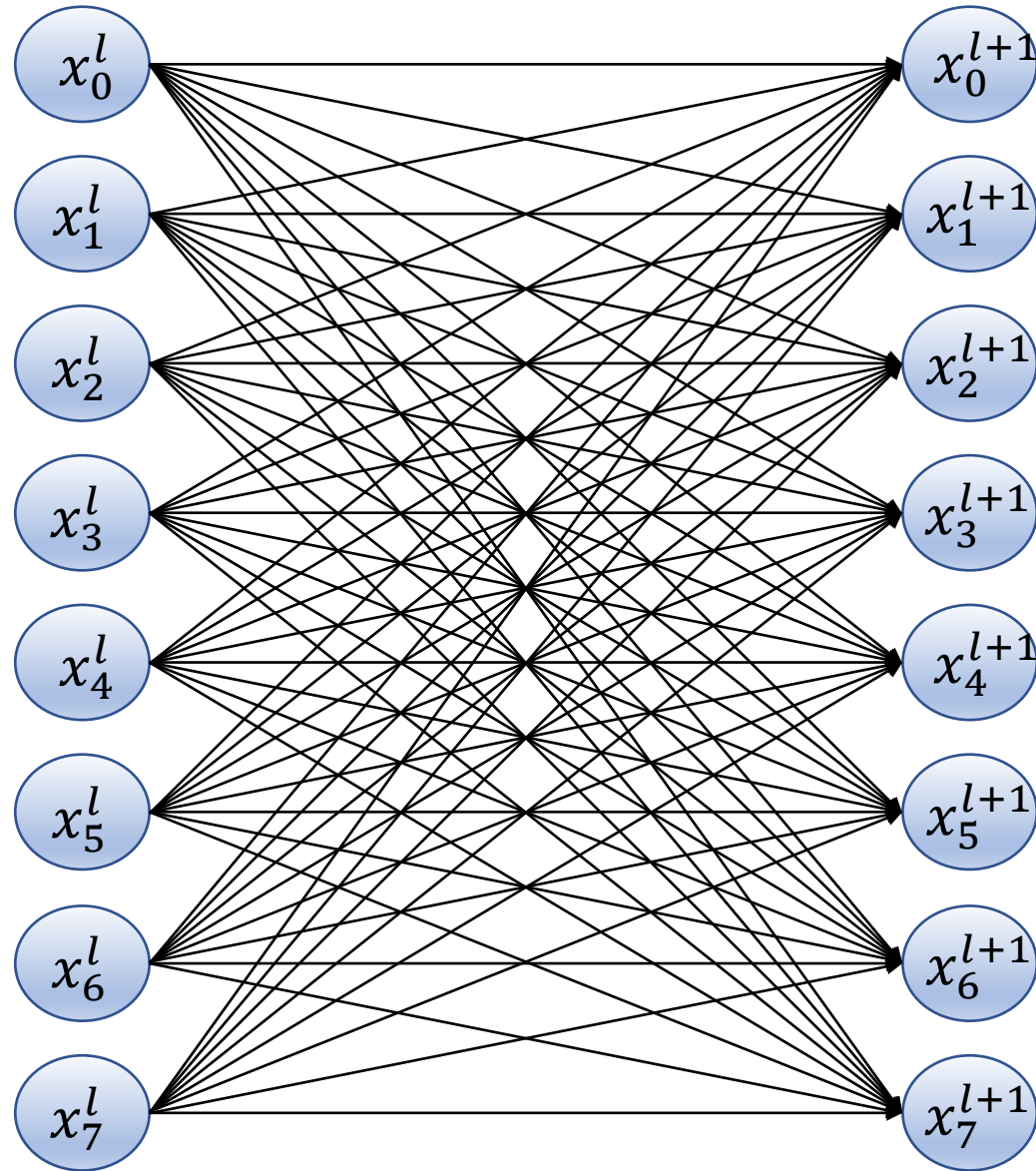
key key

horizontal
line

Expectations of visual recognition network

1. Maintain 2D structure logic
2. Shift invariant (actually, equivariant)
3. Consider only local correlations
4. Hierarchically growing field of view
5. Hierarchically progressing complexity
6. Reasonable amount of params

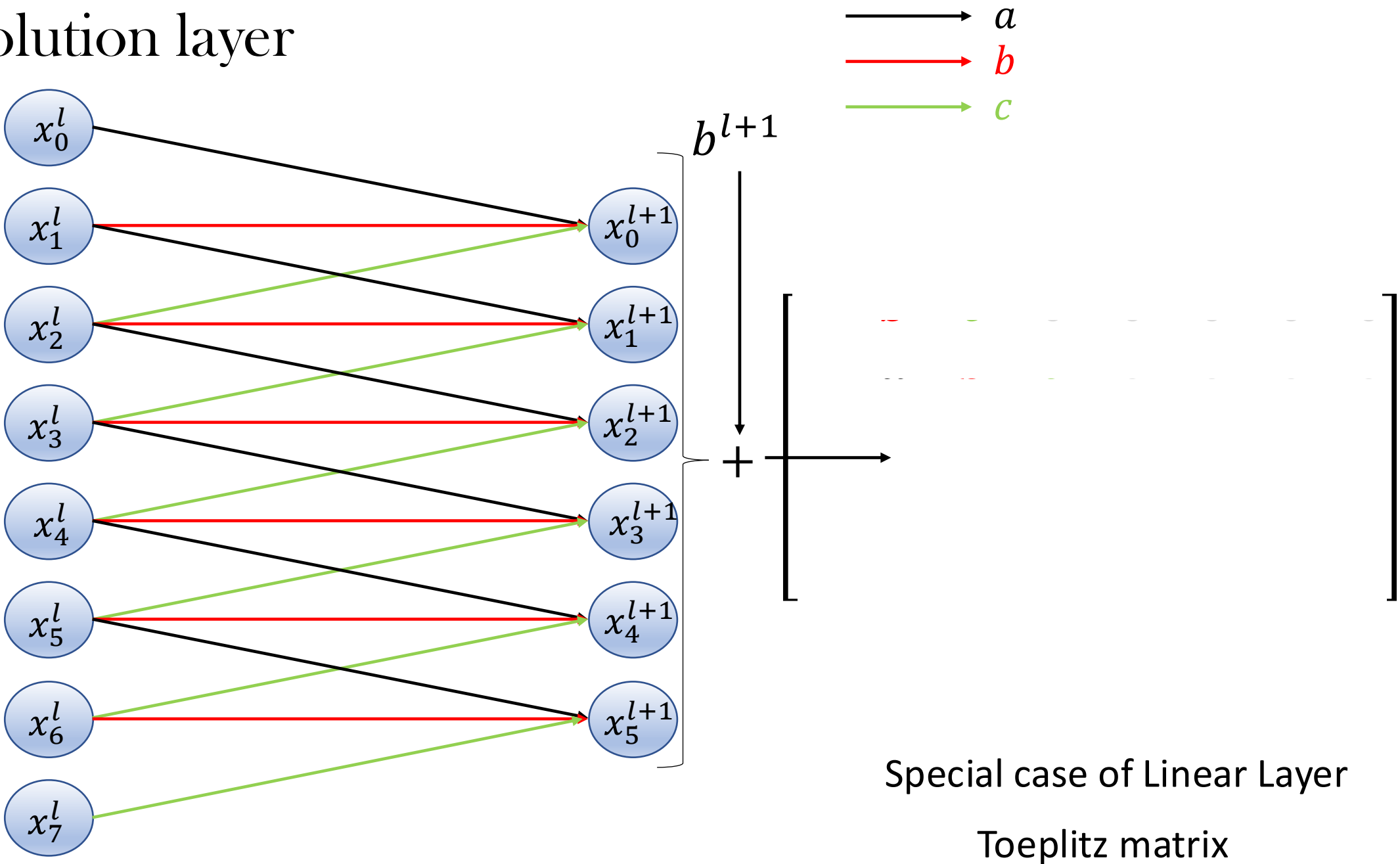
Fully connected layer



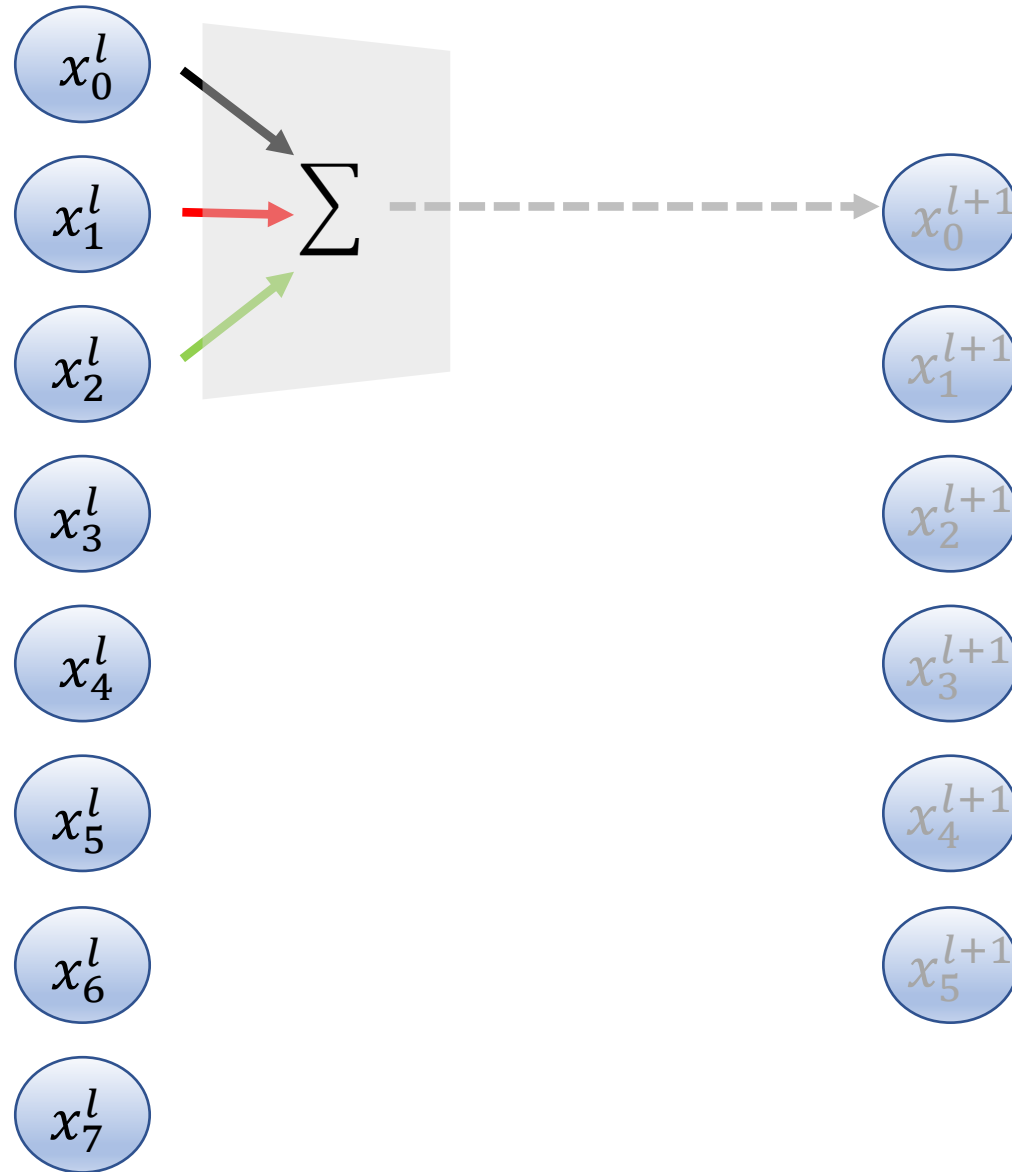
$$\begin{bmatrix} w_{00}^l & w_{01}^l & & & \\ w_{10}^l & w_{11}^l & & & \\ & & \ddots & \ddots & \\ & & & \ddots & \ddots \\ & & & & \ddots & \ddots \\ & & & & & \ddots & \ddots \\ & & & & & & \ddots & \ddots \\ & & & & & & & \ddots & \ddots \end{bmatrix}$$

- ✗ 1. Maintain 2D structure logic
- ✗ 2. Shift invariant (actually, equivariant)
- ✗ 3. Consider only local correlations
- ✗ 4. Hierarchically growing field of view
- ✓ 5. Hierarchically progressing complexity
- ✗ 6. Reasonable amount of params

Convolution layer



Convolution Filter

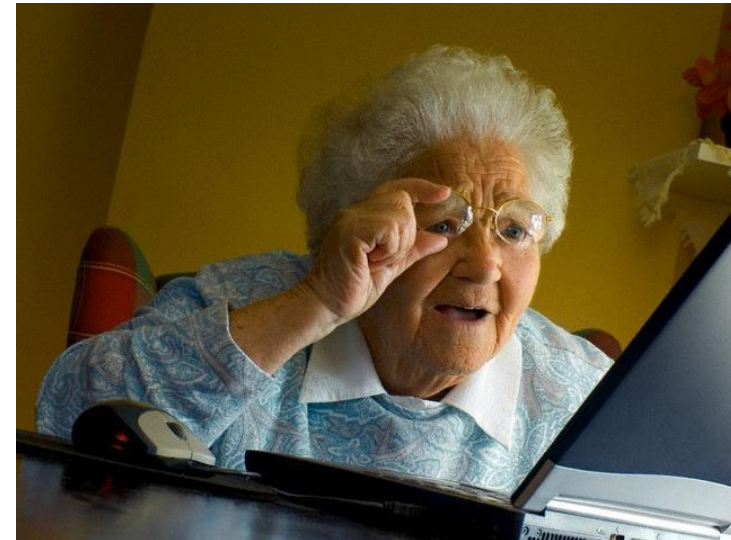


Q: Given input size N and filter size K , find output size.

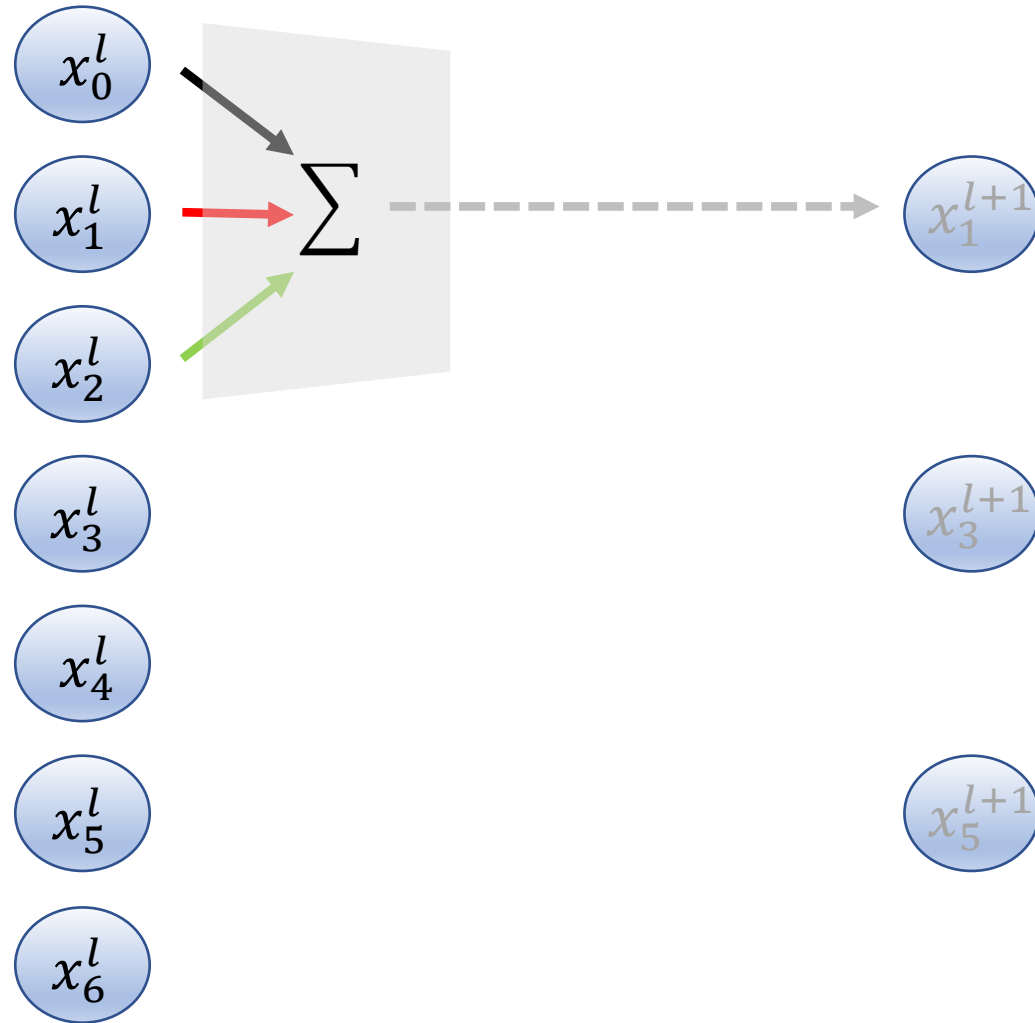
A: $N - K + 1$

Q: is this a convolution?

A: Yes, but with the flipped filter.
This is cross-correlation.



Stride

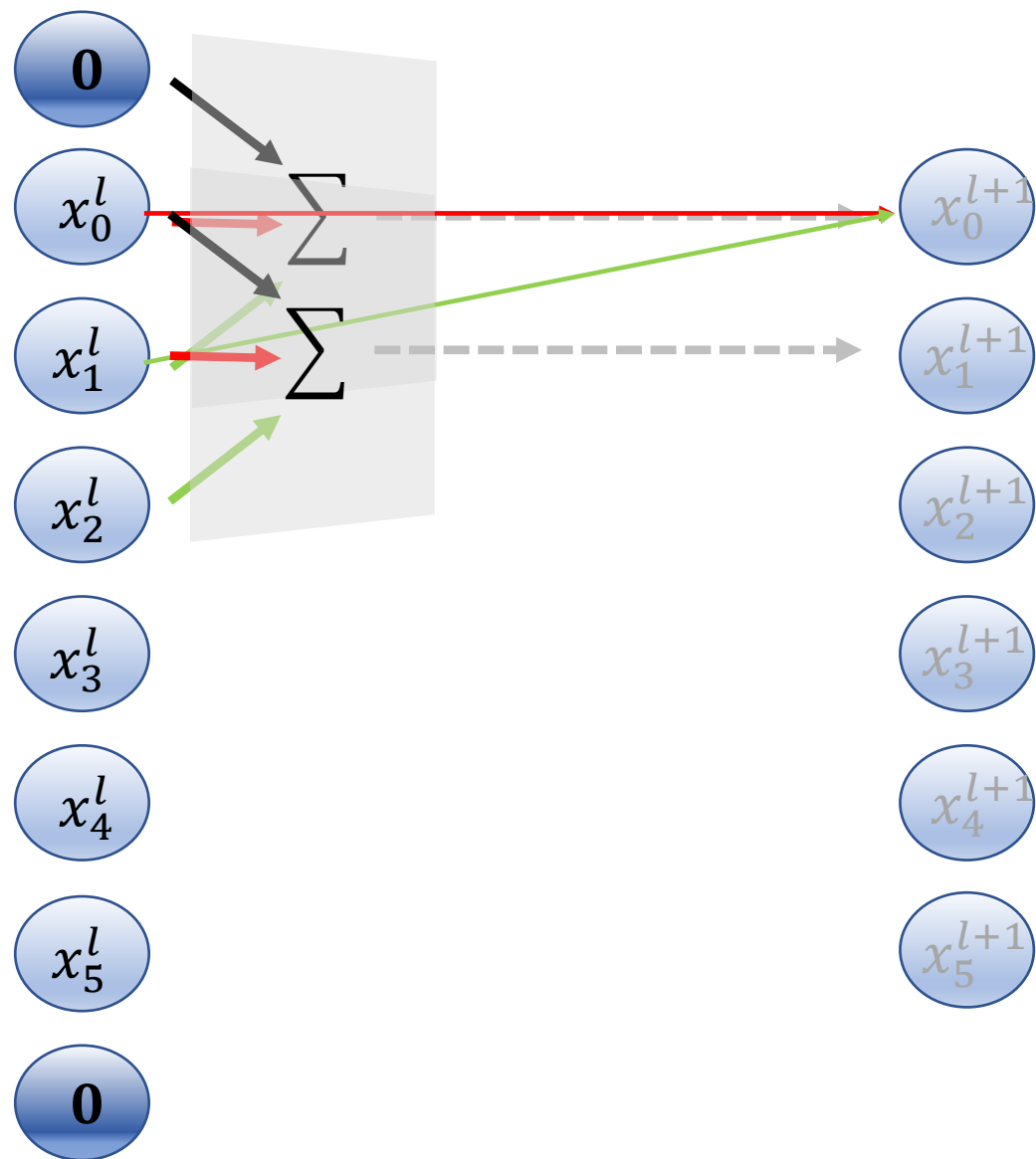


a	b	c	0	0	0	0
0	a	b	b	c	0	0
0	0	a	b	a	b	c
0	0	0	a	b	c	0
0	0	0	0	a	b	c
0	0	0	0	0	a	b

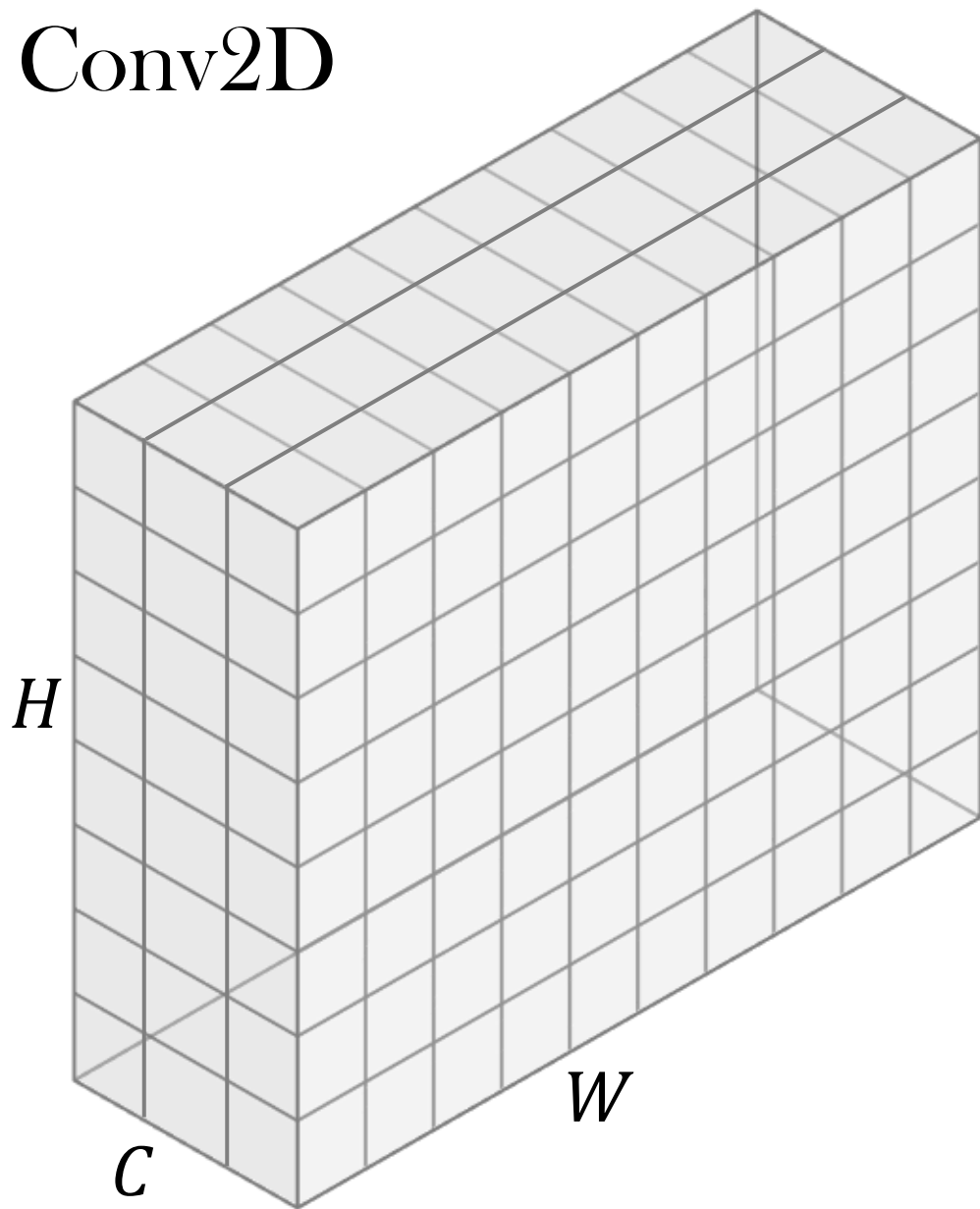
Q: Given input size N , filter size K and stride S , find output size.

A: $\left\lfloor \frac{N-K}{S} + 1 \right\rfloor$

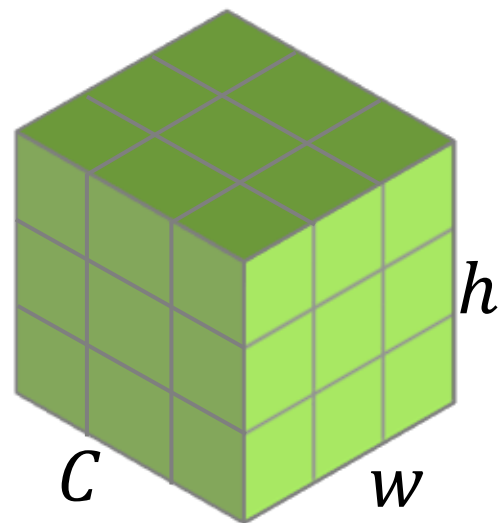
Padding



Conv2D

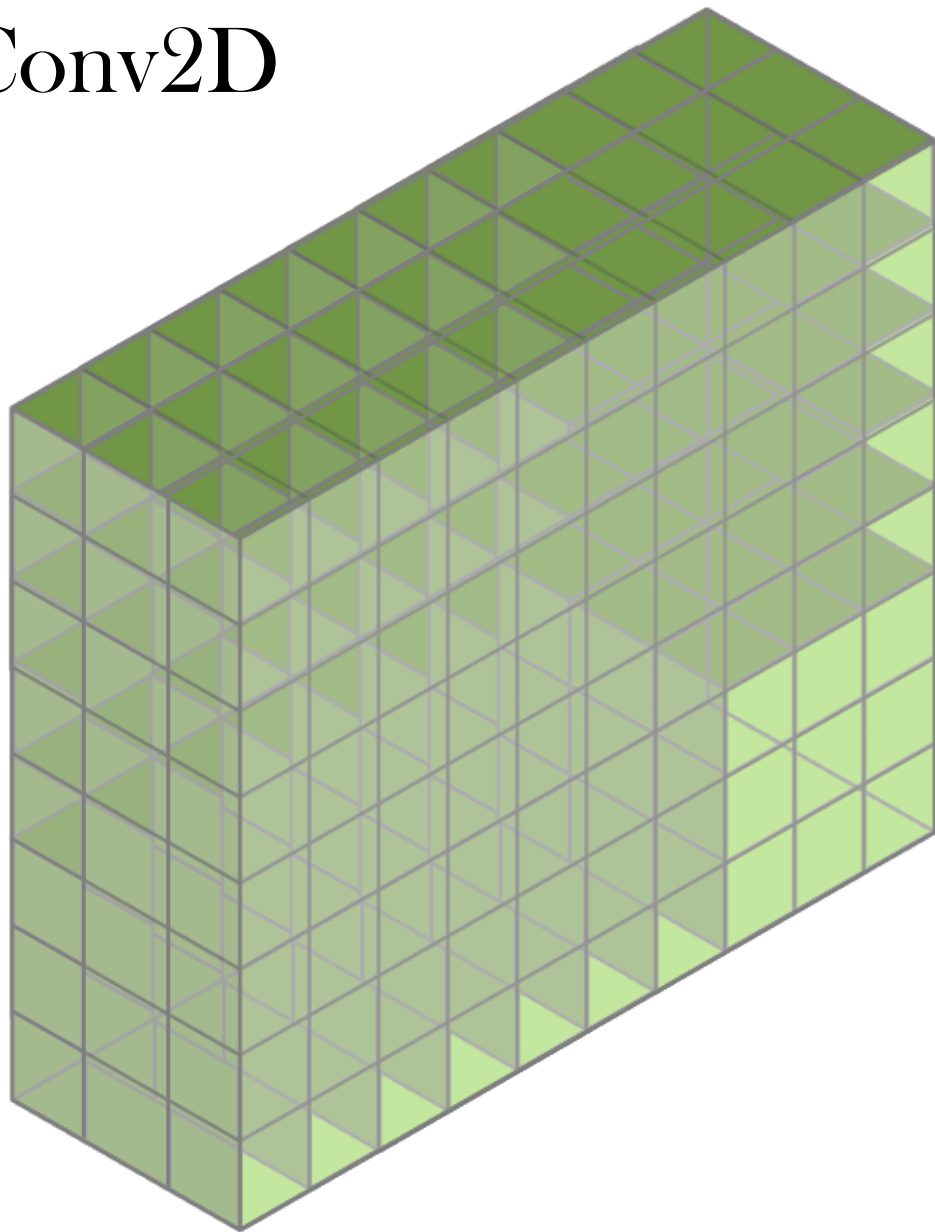


N, C, H, W

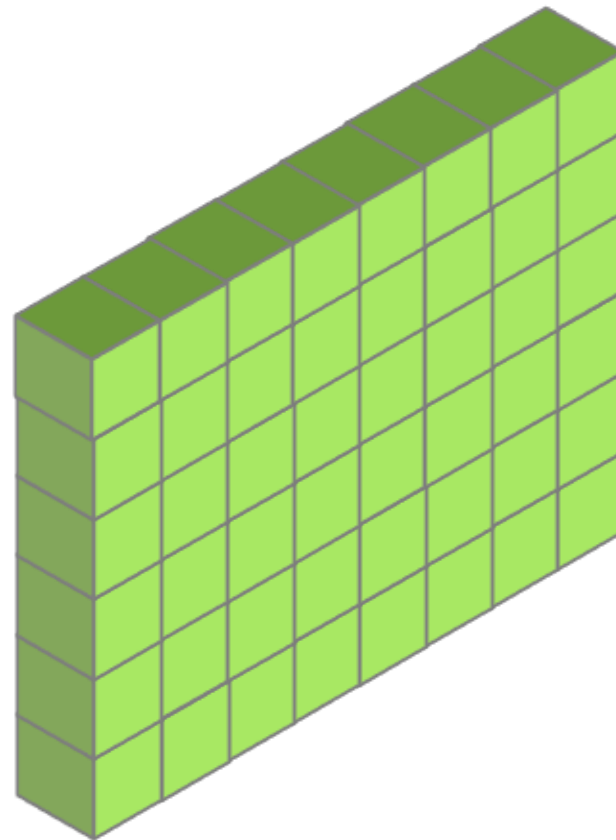


C, h, w

Conv2D

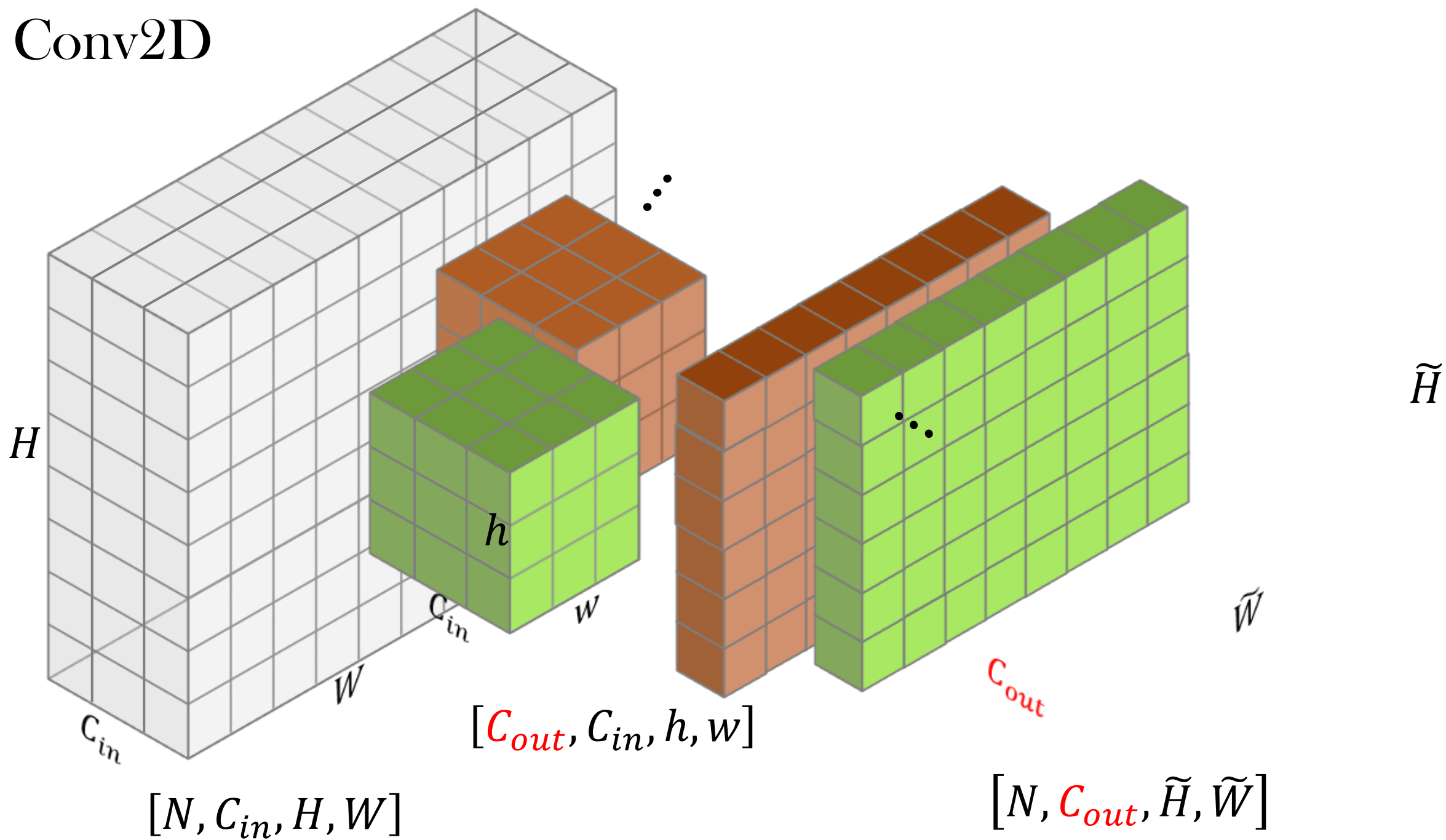


N, C, H, W



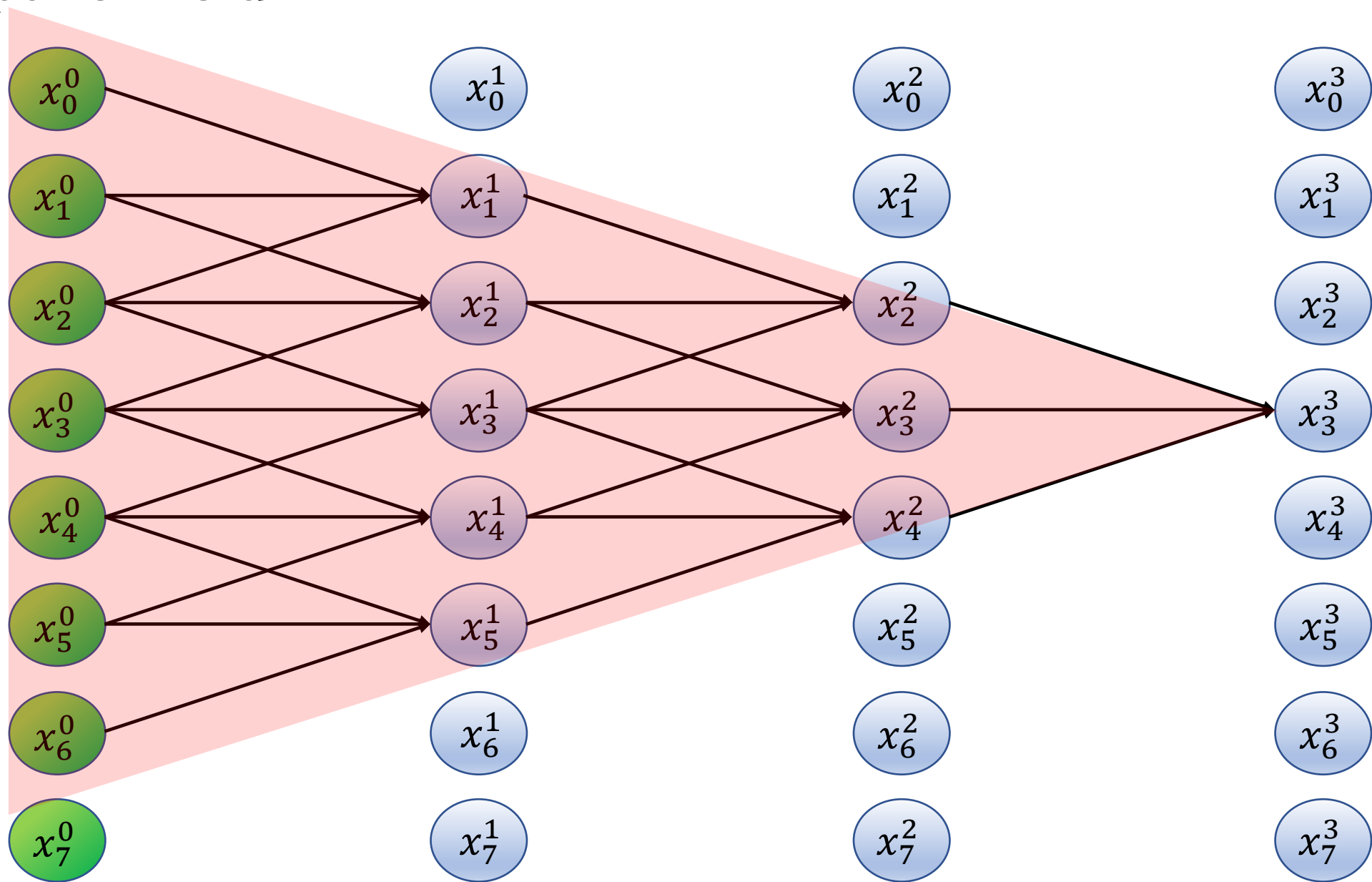
$[N, 1, \tilde{H}, \tilde{W}]$

Conv2D

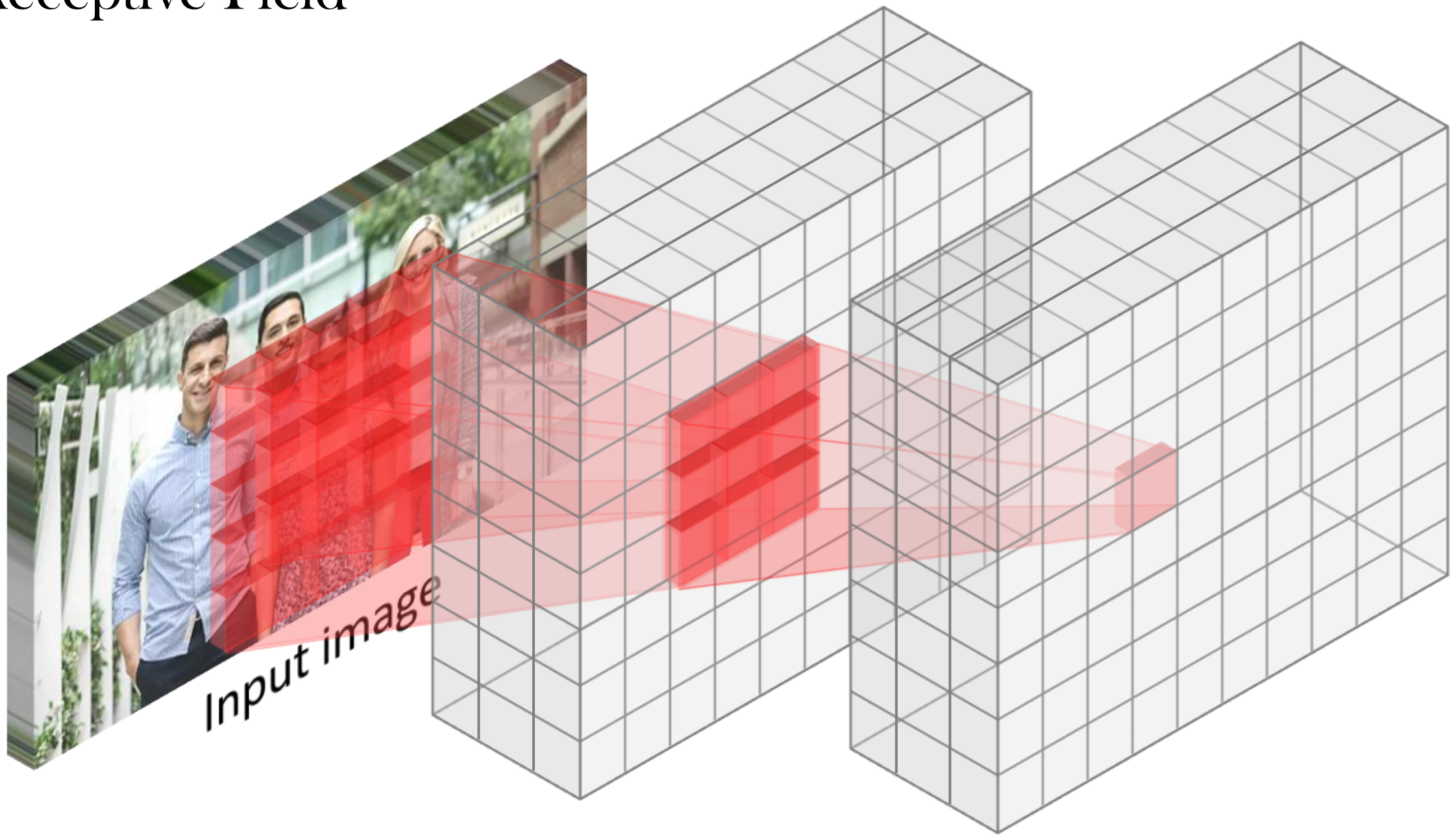


Receptive Field

[Filter size: 3]



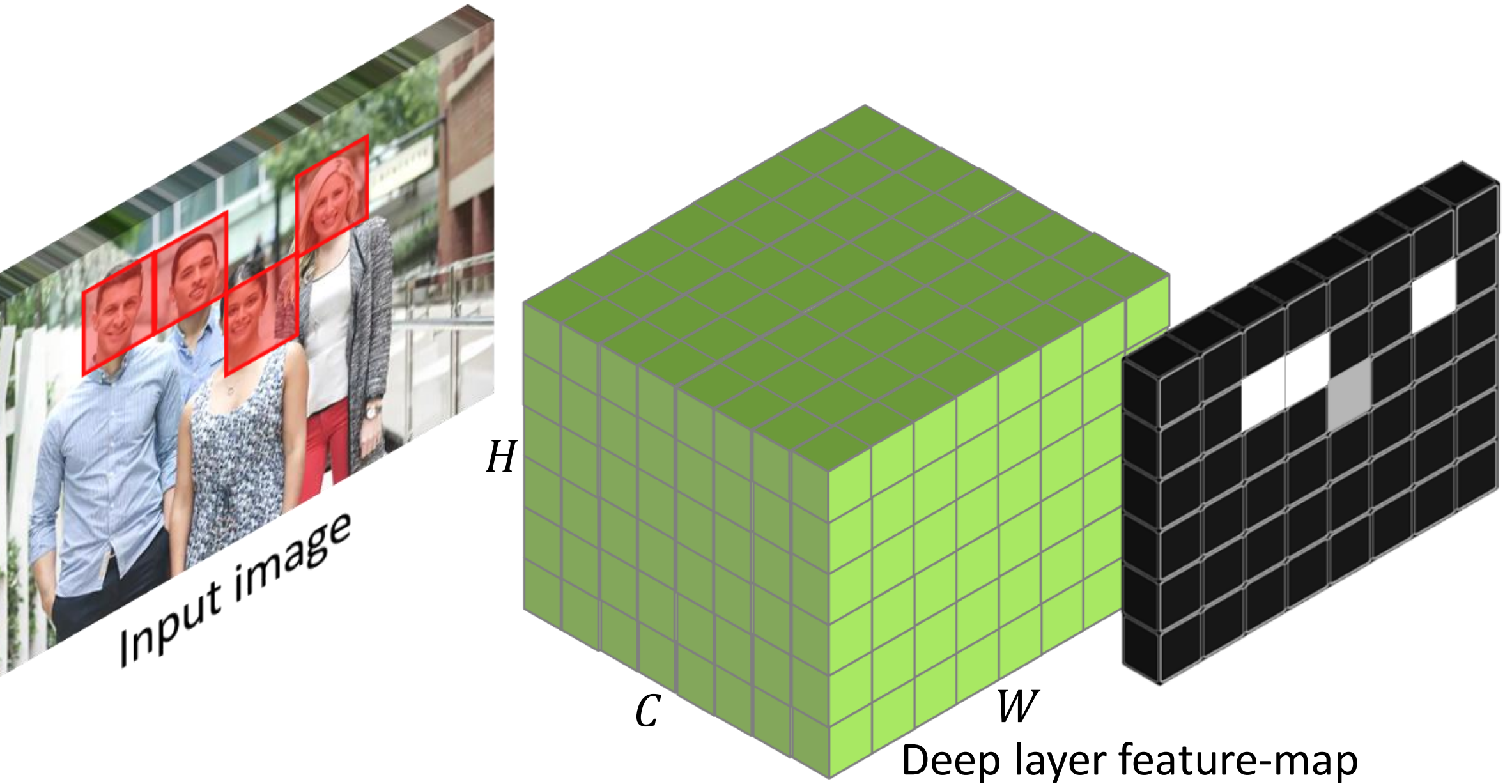
Receptive Field



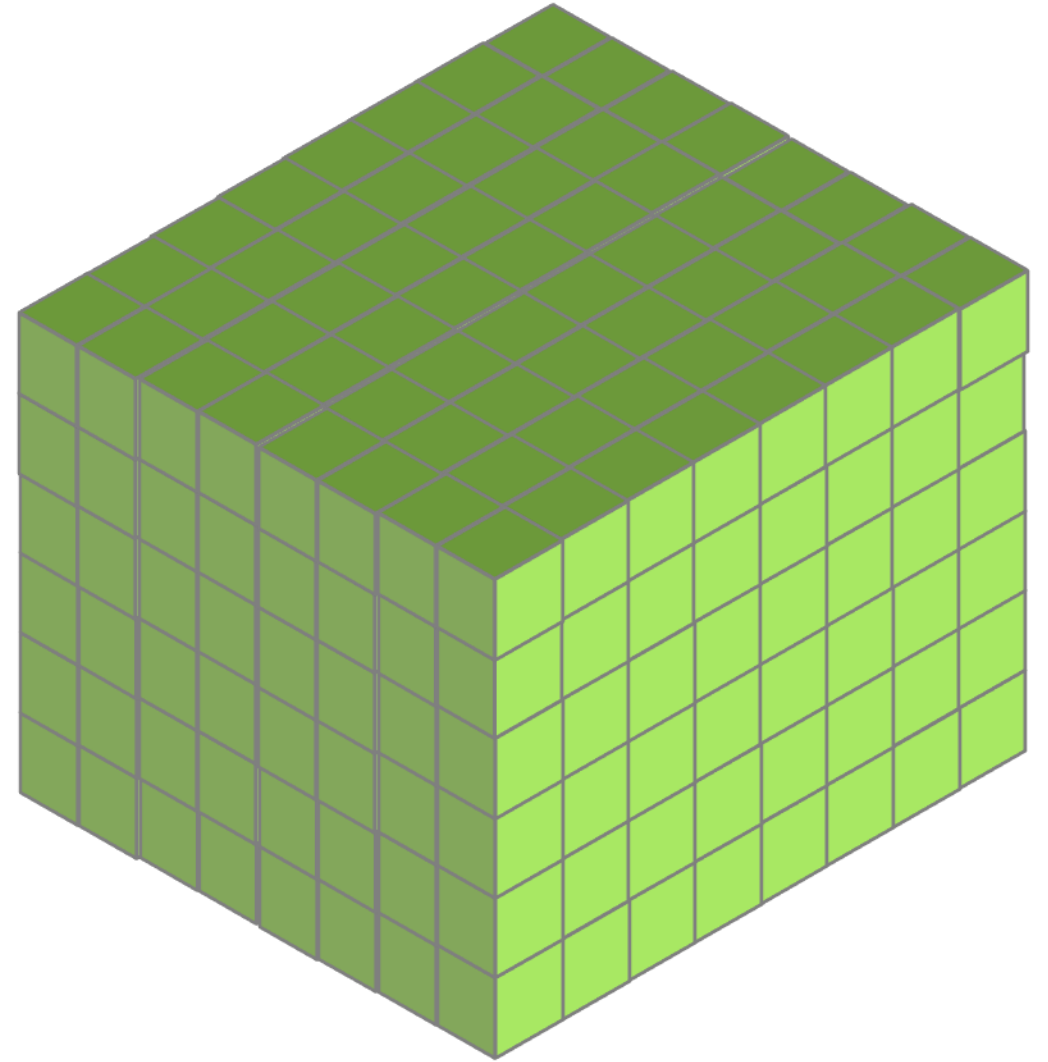
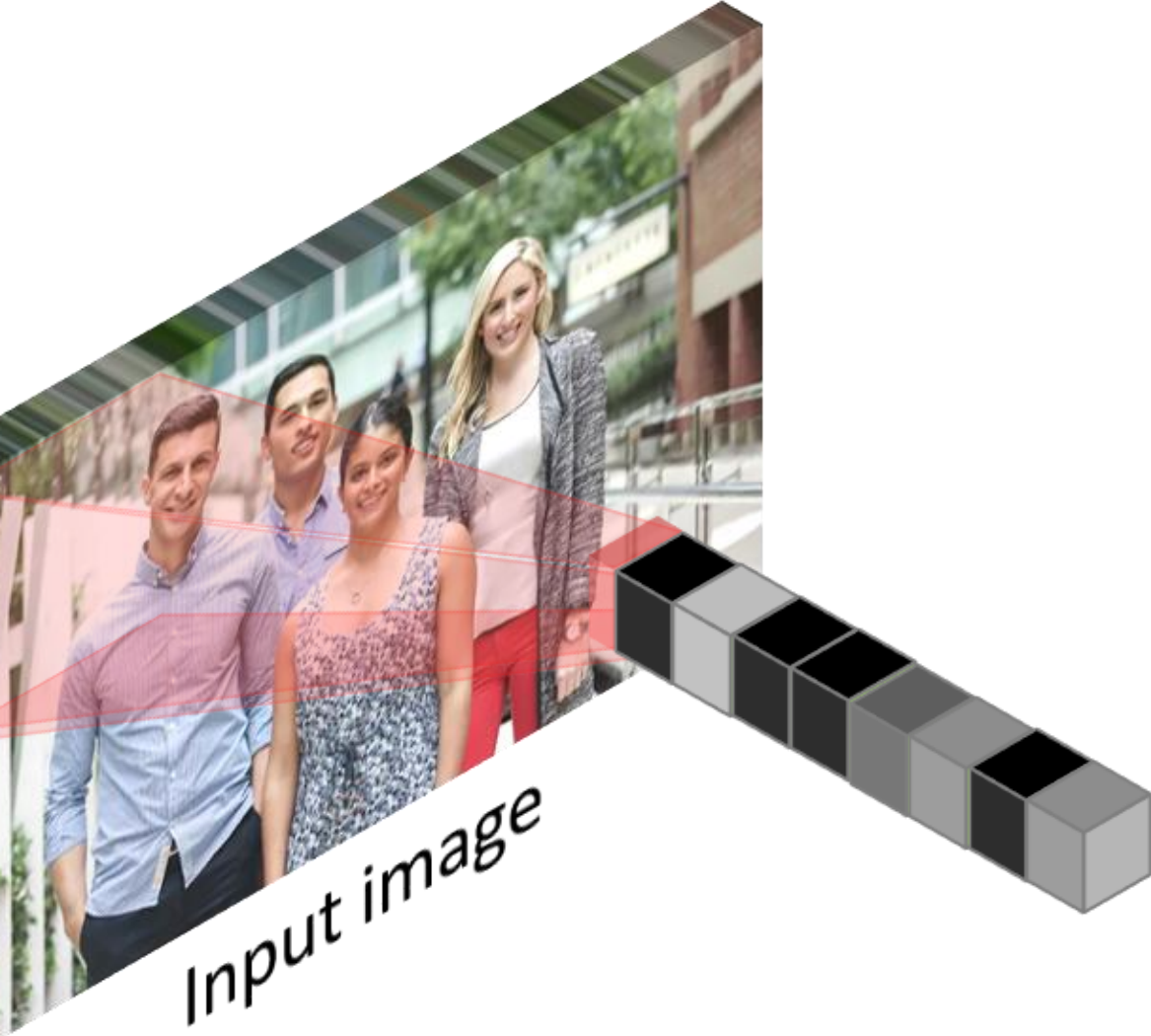
Convs rock!

- ✓ 1. Maintain 2D structure logic
- ✓ 2. Shift invariant (actually, equivariant)
- ✓ 3. Consider only local correlations
- ✓ 4. Hierarchically growing field of view
- ✓ 5. Hierarchically progressing complexity
- ✓ 6. Reasonable amount of params

Two important intuitions about feature maps

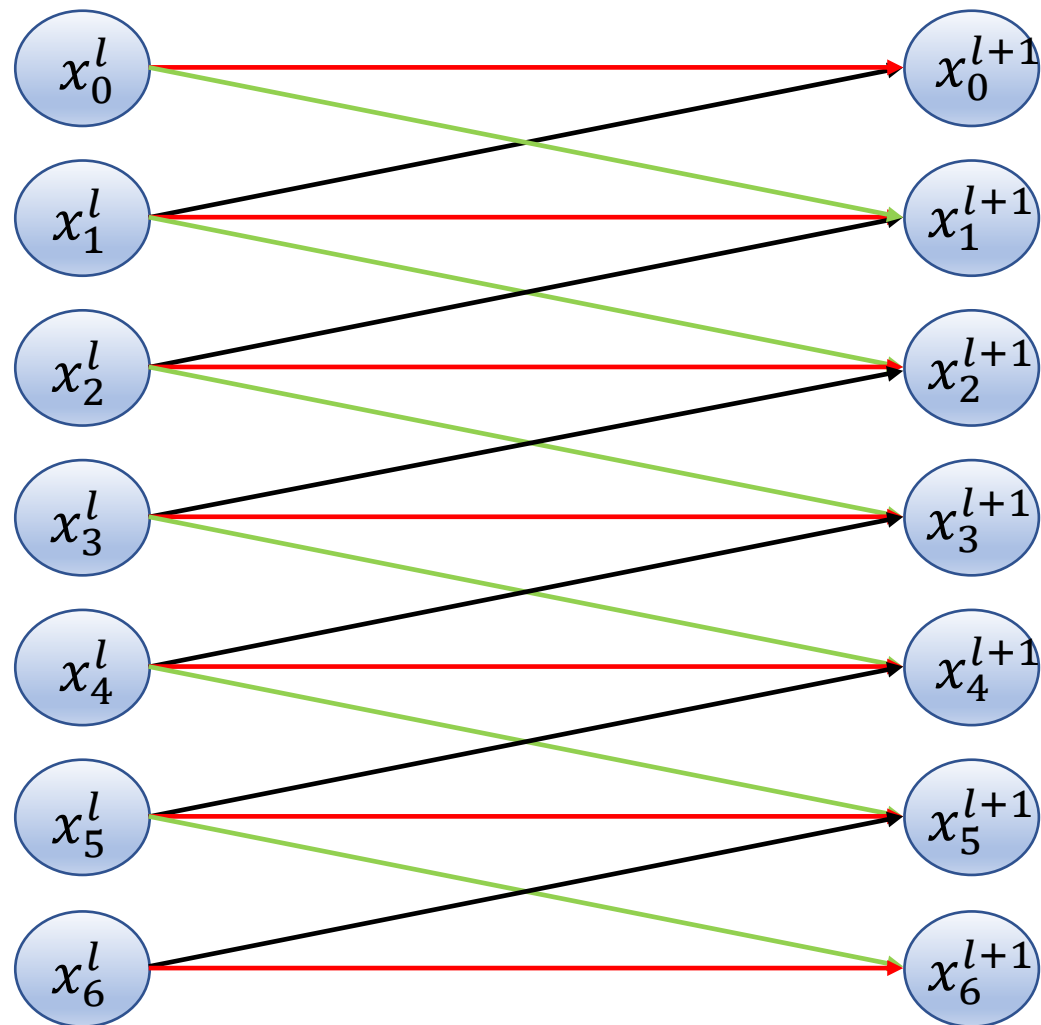


Two important intuitions about feature maps



Deep layer feature-map

Transposed Convolution



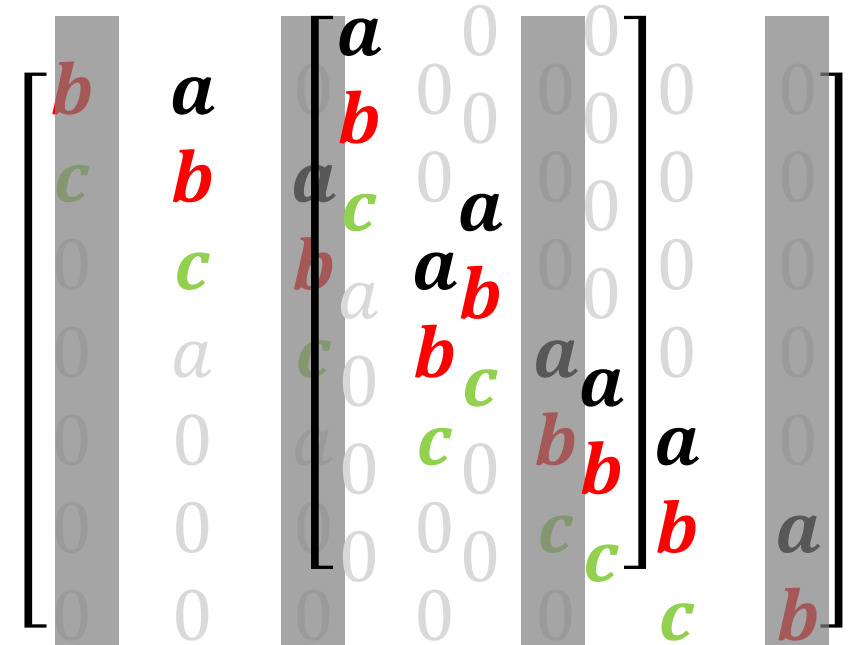
$\xrightarrow{\text{black}}$ a
 $\xrightarrow{\text{red}}$ b
 $\xrightarrow{\text{green}}$ c

$$\begin{bmatrix} \mathbf{b} & \mathbf{a} & 0 & 0 & 0 & 0 & 0 \\ \mathbf{c} & \mathbf{b} & \mathbf{a} & 0 & 0 & 0 & 0 \\ 0 & \mathbf{c} & \mathbf{b} & \mathbf{a} & 0 & 0 & 0 \\ 0 & a & \mathbf{c} & \mathbf{b} & \mathbf{a} & 0 & 0 \\ 0 & 0 & a & \mathbf{c} & \mathbf{b} & \mathbf{a} & 0 \\ 0 & 0 & 0 & 0 & \mathbf{c} & \mathbf{b} & \mathbf{a} \\ 0 & 0 & 0 & 0 & 0 & \mathbf{c} & \mathbf{b} \end{bmatrix}$$

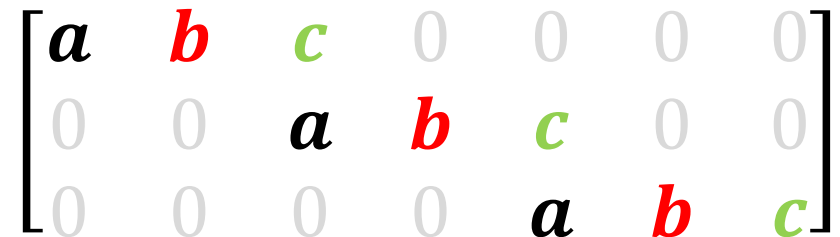
Transposed Convolution with stride



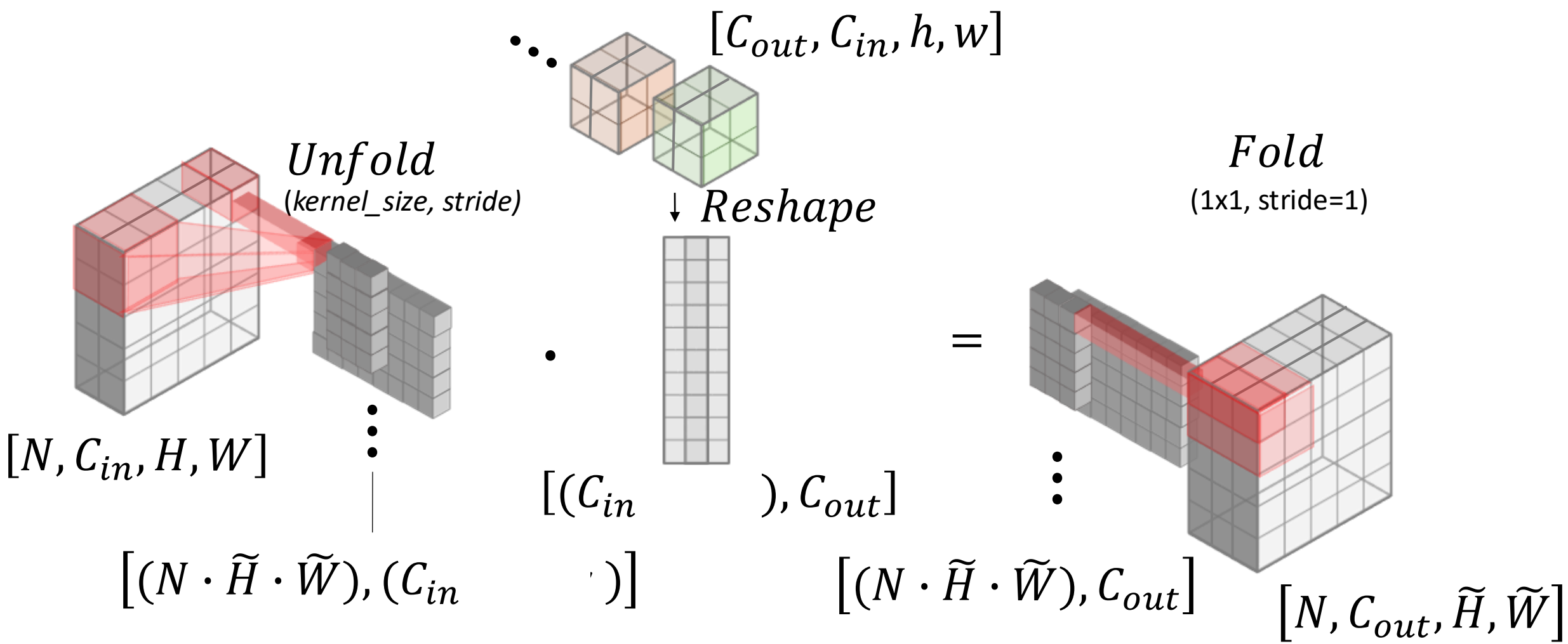
$\xrightarrow{\text{black}}$ a
 $\xrightarrow{\text{red}}$ b
 $\xrightarrow{\text{green}}$ c



Recall- stride 2 conv:



A note about the implementation of conv *TRANSPPOSED*



Q: How do you backprop a Conv? (1D, single channel)

For derivative of loss w.r.t all neurons:

Transposed Conv with the same filter!

(Conv is a special case of linear)

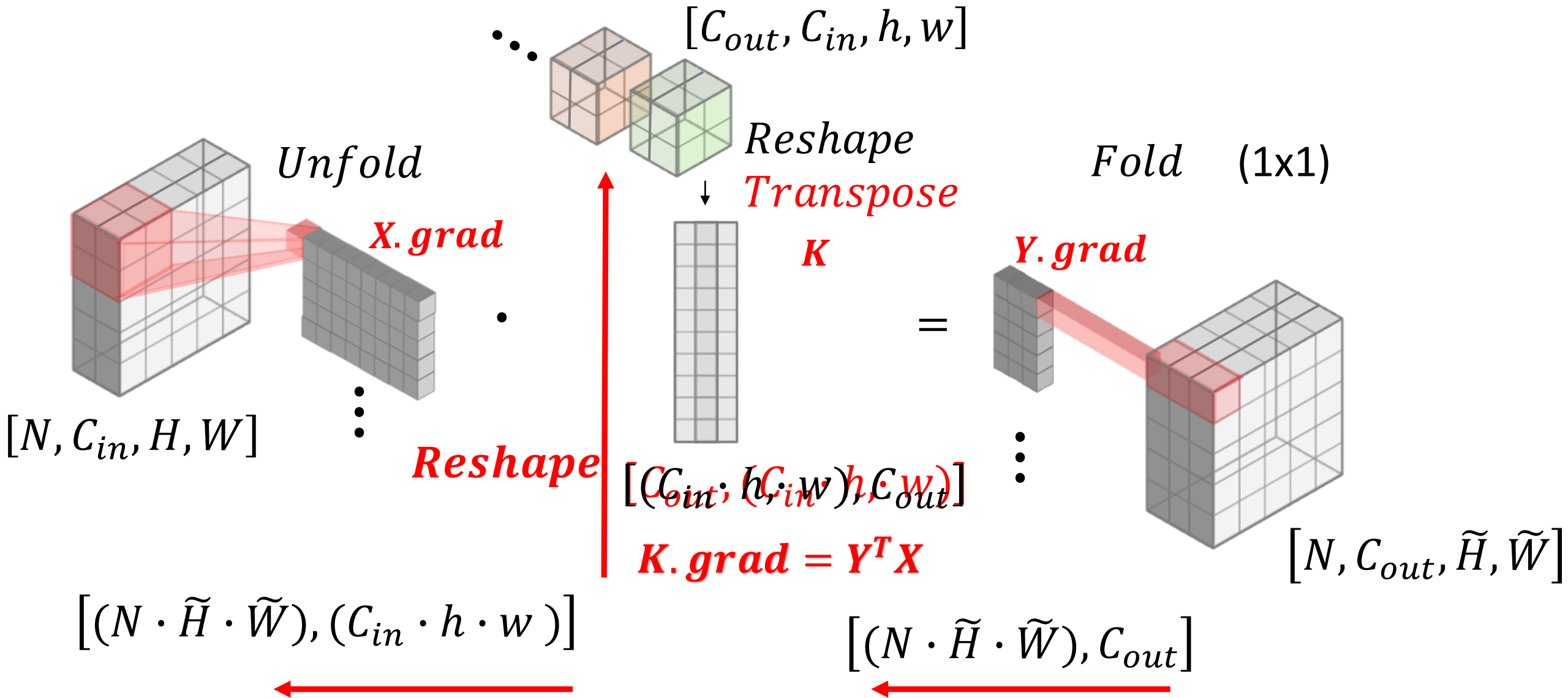
Q: How about Conv2D?

For derivative of loss w.r.t all neurons:

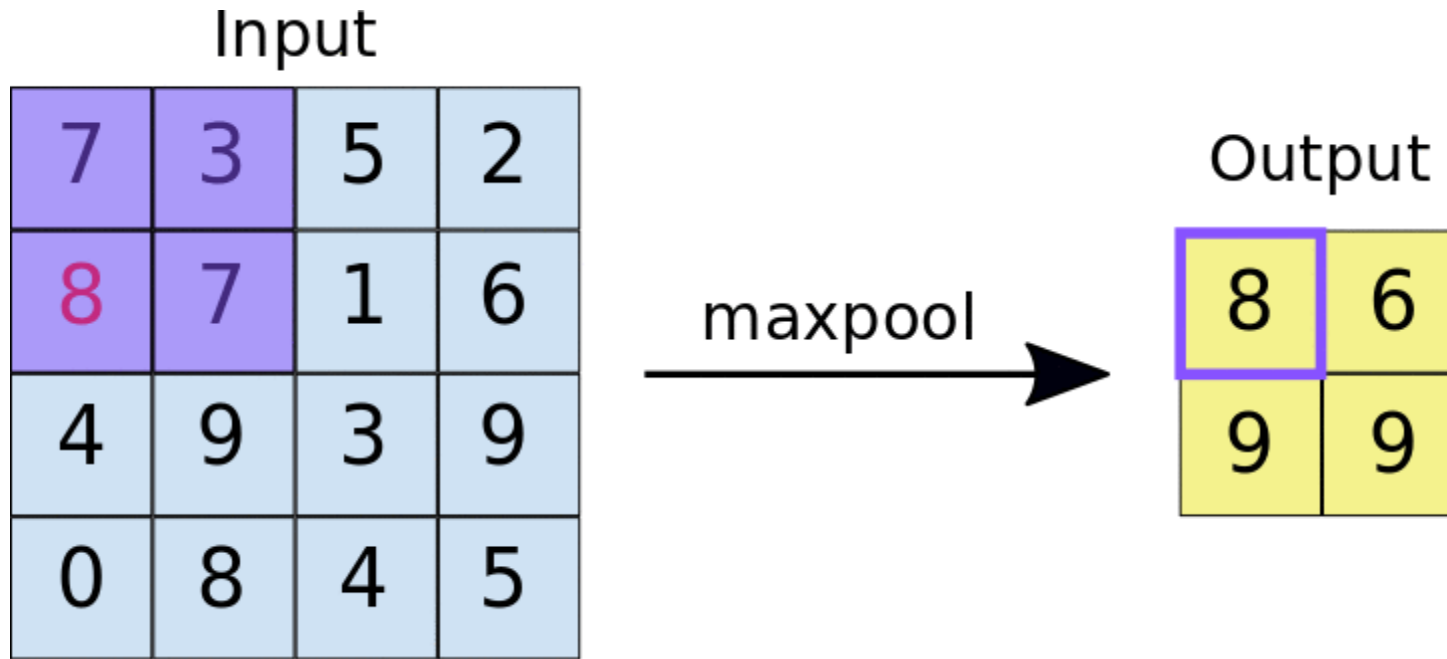
1. Transpose dims of filter c_{in} , c_{out} (as in FC)
2. Transposed Conv2D with the modified filter!

(Think of a conv as a linear layer applied to each patch separately, going from c_{in} to c_{out})

A note about the implementation of conv - BACKWARD

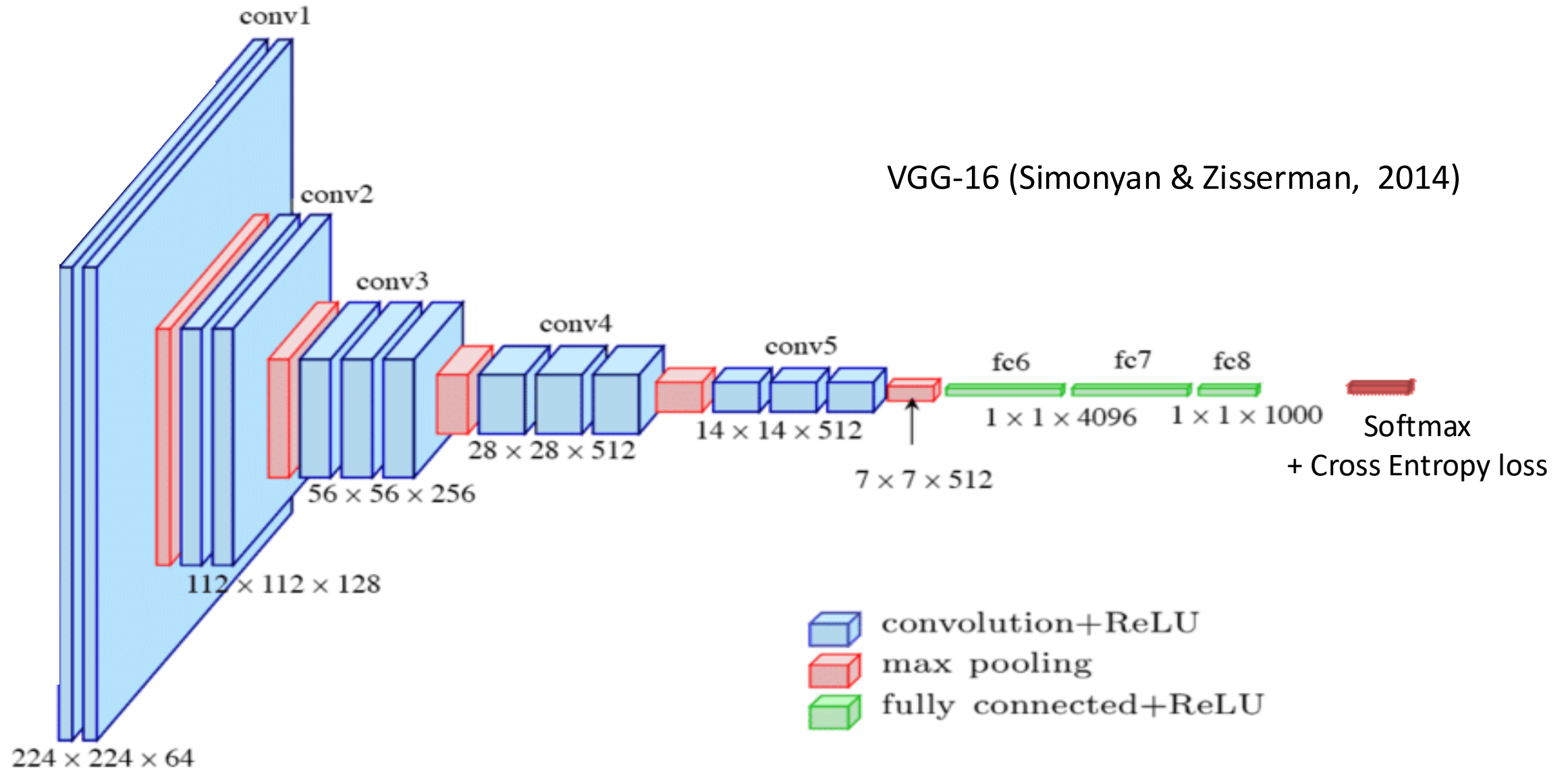


Max Pooling



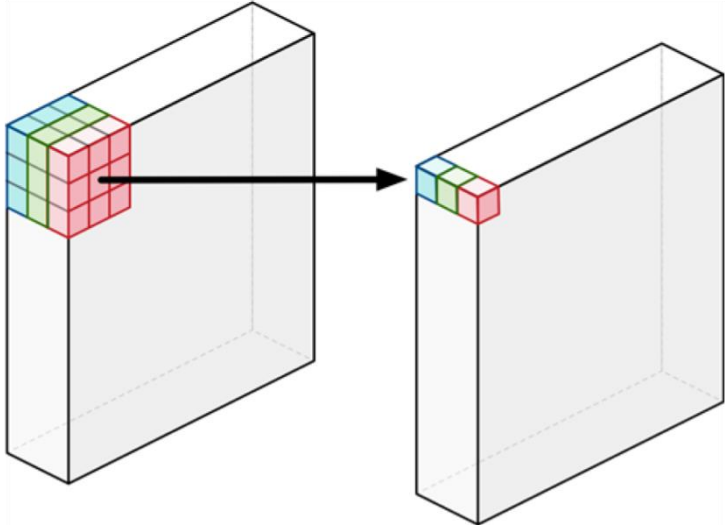
- Usually stride=win-size, but not always.
- Each channel separately.

ConvNet Example

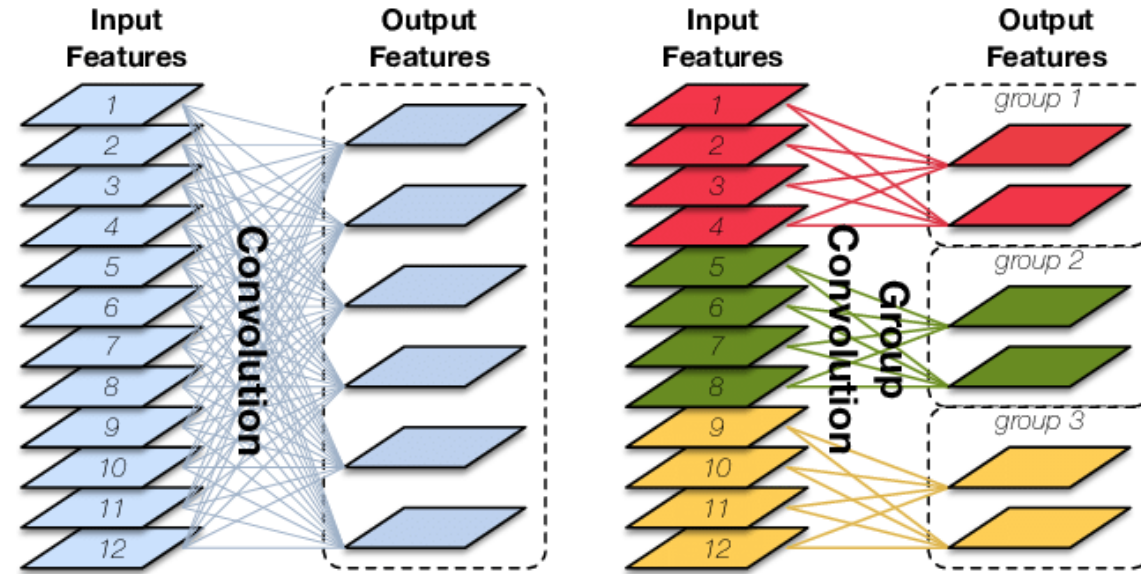


More special Convs!

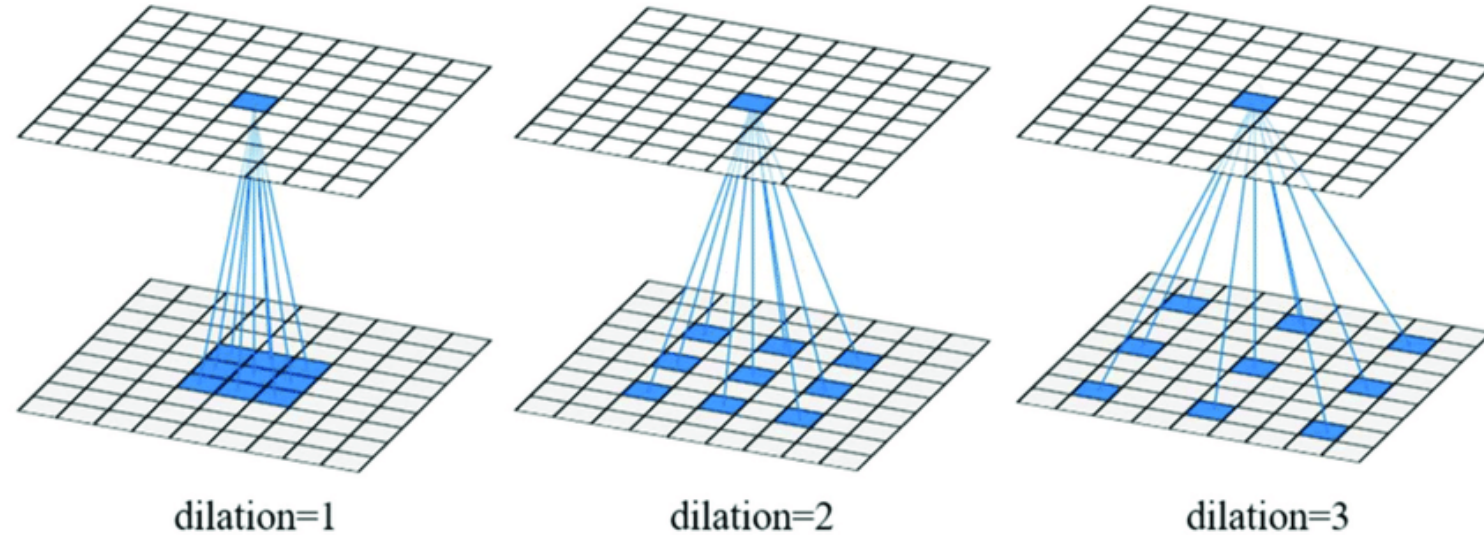
Depthwise Conv



Group Conv (Krihzevsky 2012)



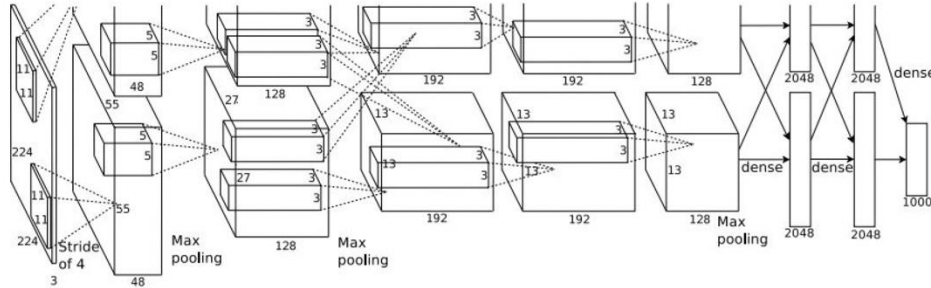
Dilated Conv (Yu&Koltun 2016)



What's going on inside ConvNets?



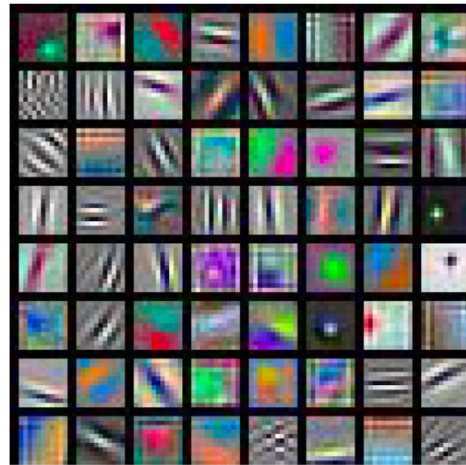
Input Image:
3 x 224 x 224



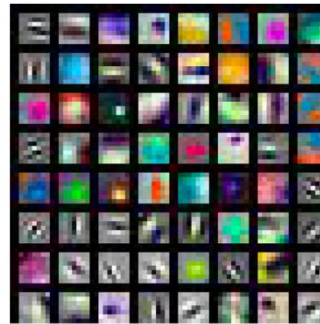
Class Scores:
1000 numbers

What are the intermediate features looking for?

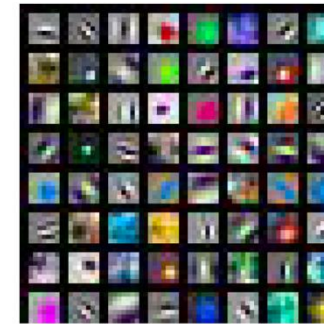
Visualize first
layer filters:



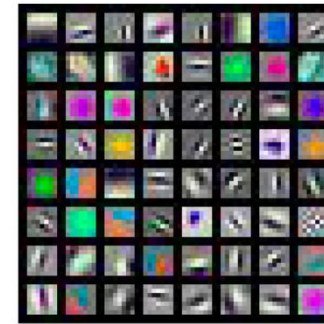
AlexNet:
64 x 3 x 11 x 11



ResNet-18:
64 x 3 x 7 x 7

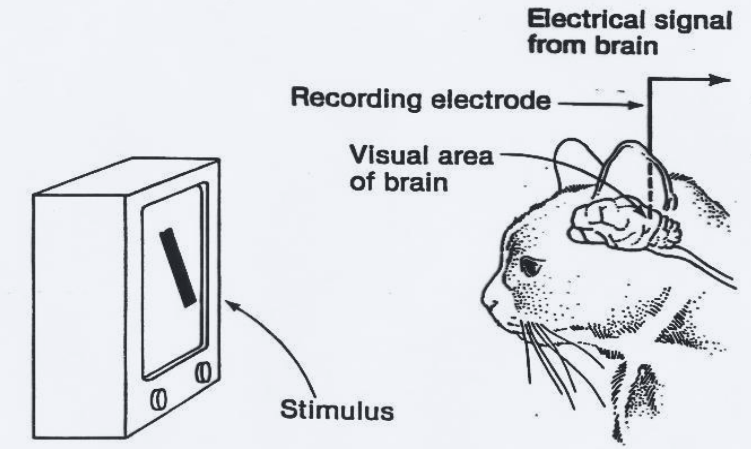


ResNet-101:
64 x 3 x 7 x 7



DenseNet-121:
64 x 3 x 7 x 7

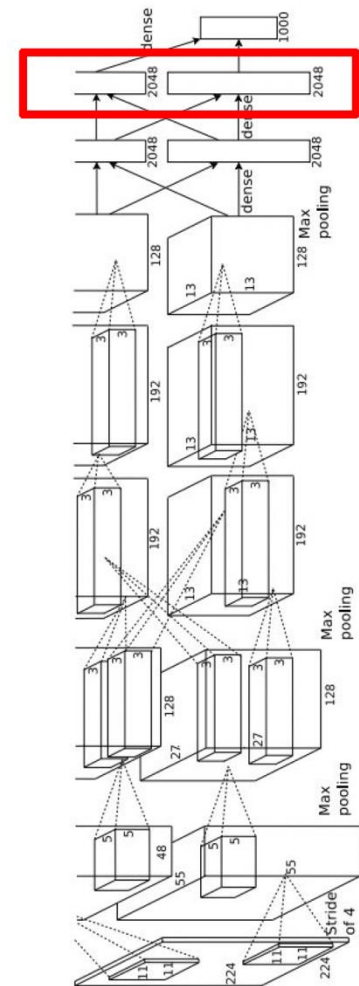
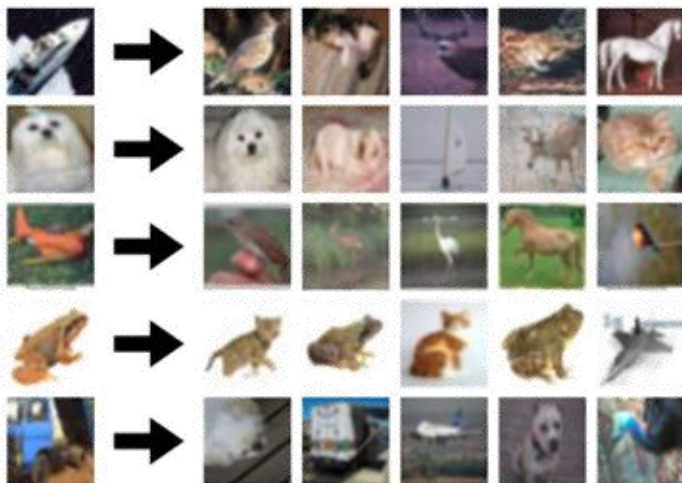
Remember Hubel & Wiesel?



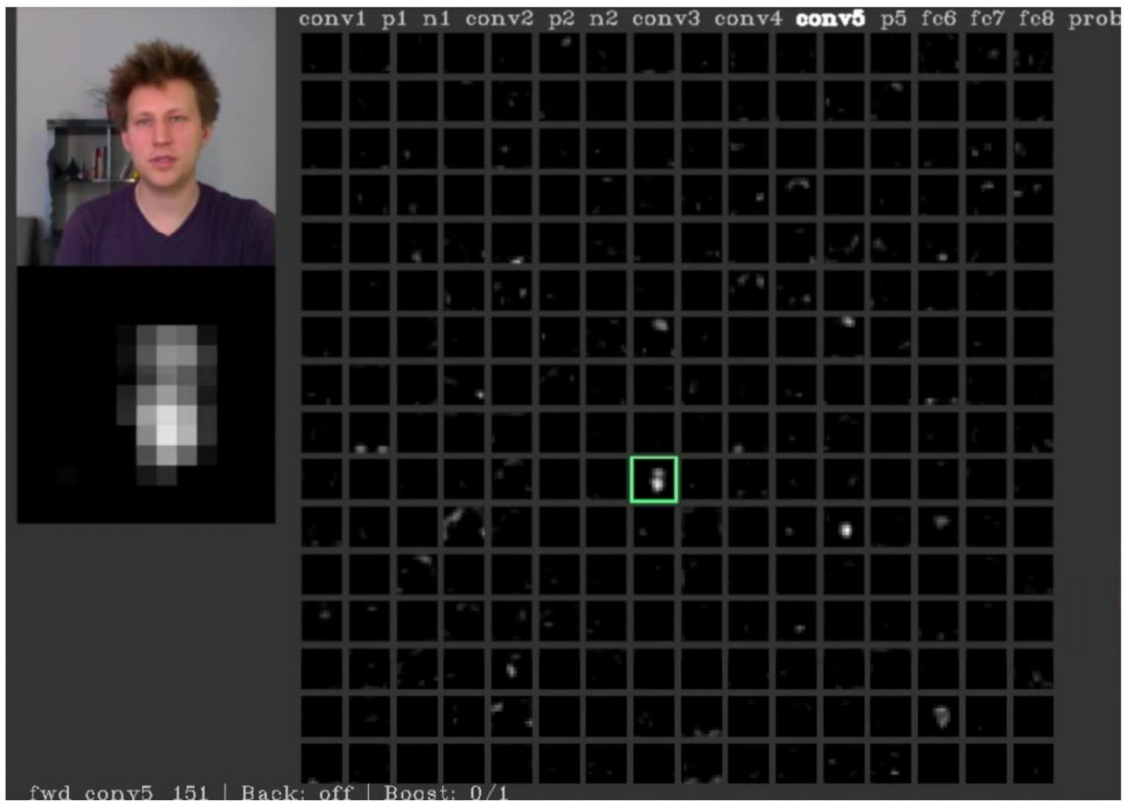
(Almost) last layer representations

Test image L2 Nearest neighbors in feature space

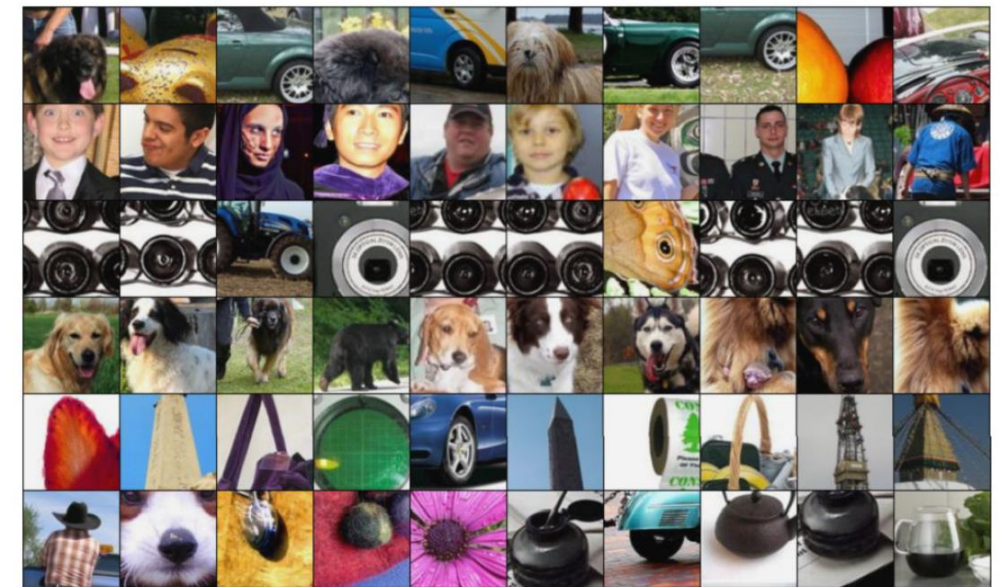
Recall: Nearest neighbors in pixel space



Activation maps

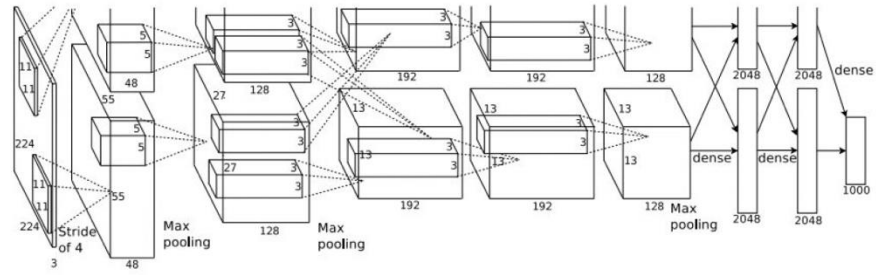
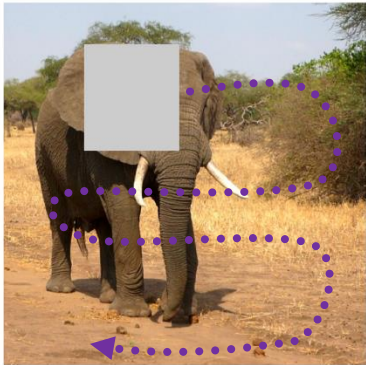


Max activations:

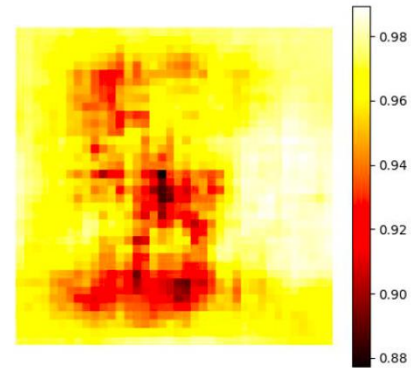


Figures from cs231n-Stanford Fei-Fei, Johnson, Yeung

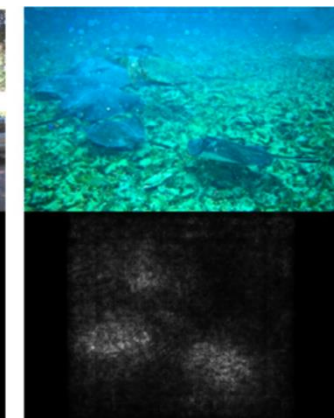
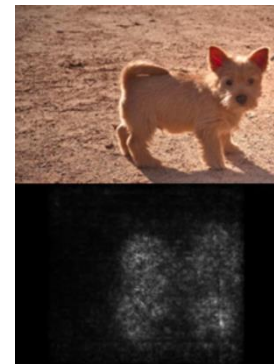
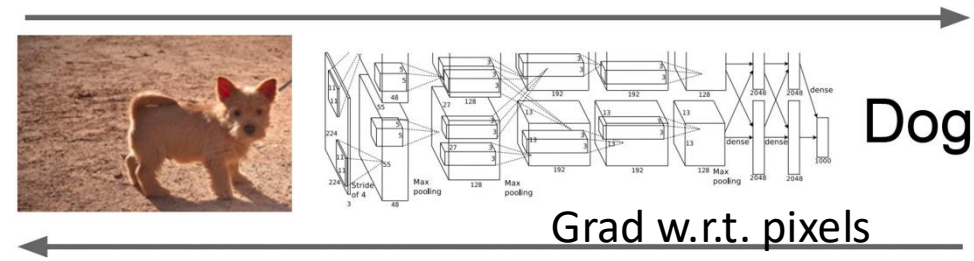
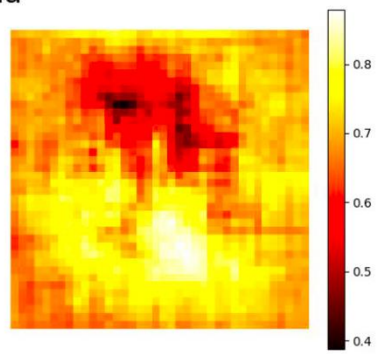
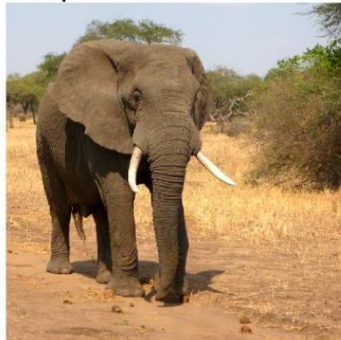
Pixel importance



schooner

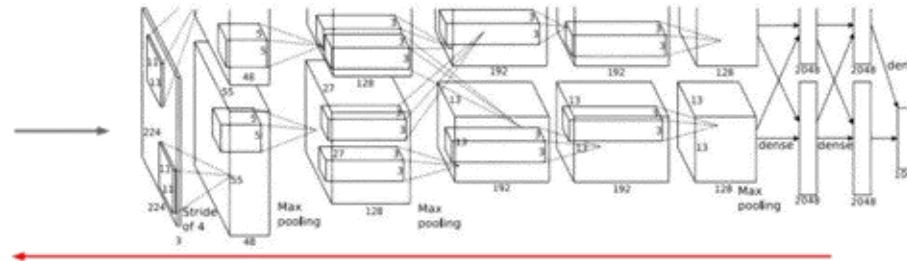


African elephant, *Loxodonta africana*



Naïve input reconstruction

1. Initialize image to zeros



$$\arg \max_I \boxed{S_c(I)} - \lambda \|I\|_2^2$$

score for class c (before Softmax)

Repeat:

2. Forward image to compute current scores
3. Backprop to get gradient of neuron value with respect to image pixels
4. Make a small update to the image



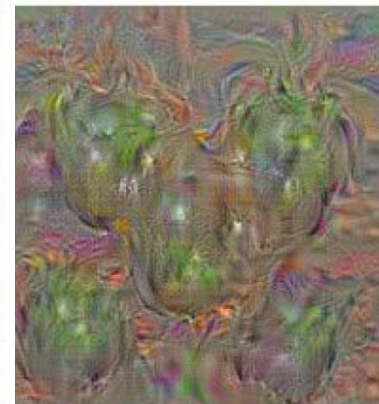
dumbbell



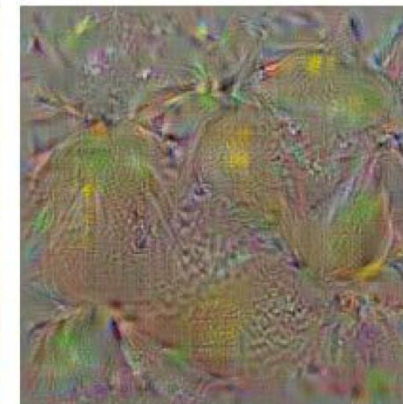
cup



dalmatian



bell pepper



lemon



husky

Thank you!

Next week's lecture:

Architectures and Practical Training

